Laboratory assignment №5

Topic: : Authentication and authorization. Web API. JSON Web Token.

Goal: acquire skills with authorization authentication mechanisms in ASP.NET, get skills working with Web API.

Workflow:

Task 1.

Implement user registration, authentication and authorization using Microsoft Identity package.

```
public IActionResult Login() => View();

[HttpPost]
[ValidateAntiForgeryToken]
0 references
public async Task<IActionResult> Login(string login, int password)
{
    var user = repository.Users.FirstOrDefault(u => u.Login == login && u.Password == passwor
    if (user == null)
    {
        ModelState.AddModelError(string.Empty, "Невірний логін або пароль");
        return View();
    }

    var claims = new List<Claim>
    {
```

Fig 1 – AccountController/Login

```
0 references
public IActionResult Register()
{
    return View();
}

[HttpPost]
[ValidateAntiForgeryToken]
0 references
public IActionResult Register(Users user)
{
    if (repository.Users.Any(u => u.Login == user.Login))
    {
        ModelState.AddModelError("Login", "Користувач з таким логіном вже існує");
    }
```

Fig 2 – AccountController/Register

Additional points task (5 points): implement work with user roles.
Provide the possibility of registering two types of users in the project, for example, a patient and a doctor. If the project topic does not involve working with several types of users, then implement the roles of a regular user and an administrator.

Difference between roles “manager”, “employee” and “admin”

```
public IActionResult Index()
{
    var userId = long.Parse(User.FindFirst(ClaimTypes.NameIdentifier).Value);
    var userRole = User.FindFirst(ClaimTypes.Role).Value;

    if (userRole == "Manager" || userRole == "Admin")
    {
        return View(repository.Users.ToList());
    }
    else if (userRole == "Employee")
    {
        var currentUser = repository.Users.FirstOrDefault(u => u.UsersID == userId);
        return View(new List<Users> { currentUser });
    }

    return Forbid();
}
```

Fig 3 – Roles and access politics

Task 2.

Create a new WebAPI project in your solution. This project should implement the functionality of your MVC project as REST endpoints. For better code organization it is recommended to move classes for interacting with the database (models, contexts, repositories) to the separate Class Library project and add reference to this library in both MVC and WebAPI projects.
After this, configure ConnectionStrings for the created databases inside the WebAPI project appsettings.json file. Thus, after moving the files for the interaction with the database to a separate library, we can reuse all the necessary services in both projects.

Implement all the necessary CRUD-operations in your WebAPI project, including user registration. Test your endpoints using Postman, Swagger or any other suitable tool.

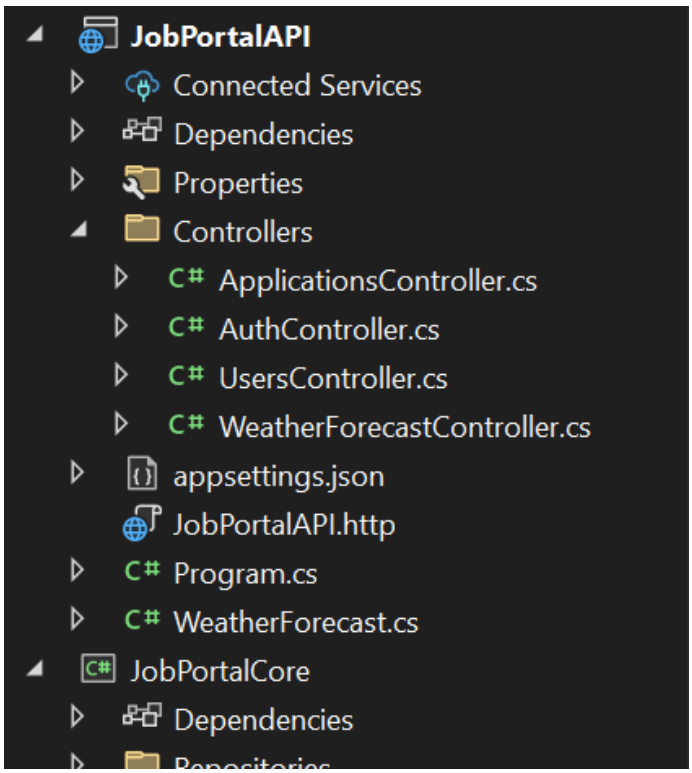
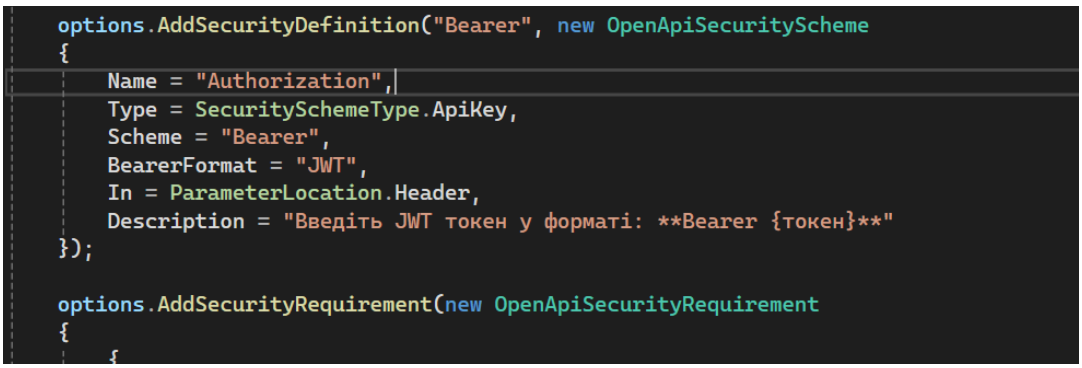


Fig 4 – webAPI with CRUD operations

Task 3.

Implement user authentication and authorization using the JWT mechanism. It should work the following way:

1. The client sends a request to the login endpoint with user login and password.
2. If a user with such a login and password exists, then the server must return the generated JSON Web Token.
3. To gain access to actions that require authorization, the client sends an Authorization header in the request with the value Bearer TokenValue, where TokenValue is the token that was generated during authentication.
4. If the token is valid, the server will execute the request, otherwise it will return a 401 Unauthorized error.



```
options.AddSecurityDefinition("Bearer", new OpenApiSecurityScheme
{
    Name = "Authorization",
    Type = SecuritySchemeType.ApiKey,
    Scheme = "Bearer",
    BearerFormat = "JWT",
    In = ParameterLocation.Header,
    Description = "Введіть JWT токен у форматі: **Bearer {токен}**"
});

options.AddSecurityRequirement(new OpenApiSecurityRequirement
{
    {
```

Fig 5 – JWT-authorization

Conclusions: During the laboratory work, I acquire skills with authorization authentication mechanisms in ASP.NET, get skills working with Web API.